MASTER D'EXCELLENCE

SYSTEMES REPARTIS ET DISTRIBUES

INGENIERIE INFORMATIQUE ET SYSTEMES EMBARQUES

Rapport de projet de fin de semestre

Intitulé du Sujet :

Surveillance de la température d'un entrepôt

Réalisé par :

BAADI Anass

EL-ATIKI Yassine

ASKLOU Abdessamad

AIT GOURRAMT Abdelhamid

AIT MOULAY BRAHIM Noureddine

Encadré par :

Pr. IDRAIS Jaafar

Dr. RIDA Zakaria

A.U 2024/2025

TABLE DES MATIERES

TABLE DES MATIERES **1**

TEBLE DES FIGURES **3**

RESUME **4**

ABSTRACT **5**

INTRODUCTION GENERALE **6**

CHAPITRE 1 : CONTEXTE GENERAL DU PROJET **7**

I. INTRODUCTION : 7
II. GENERALITES SUR LE PROJET : 7
1. Contexte : 7
2. Objectif : 7
III. LES FONCTIONNALITES DE SYSTEME : 7
IV. LES DIFFERENTES CLASSES D'USAGERS DU SYSTEME : 8
1. Utilisateur : 8
V. L'ETUDE DE L'EXISTANT : 8
VI. SOLUTION : 8
VII. CONCLUSION : 8

CHAPITRE 2 : ANALYSE ET CONCEPTION DE SYSTEME **9**

I. INTRODUCTION : 9
II. DIAGRAMMES UML : 9
1. Diagramme de classe : 9
2. Diagramme de déploiement : 10
3. Diagramme de séquence : 11
4. Diagramme des cas d'utilisation : 12
III. CONCLUSION : 13

CHAPITRE 3 : REALISATION DE SYSTEME **14**

I. INTRODUCTION : 14
II. LES OUTILS UTILISES : 14
1. Visual Studio Code : 14
2. Spring Boot : 14
3. JAVA RMI : 15
4. MQTT : 15
5. WOKWI : 15
6. XAMPP : 16
7. MySQL : 16

8. GitHub : 16
III. FONCTIONNEMENT DE PROJET : 17
1. Démarrage de XAMPP : 17
2. Démarrage de projet (Spring Boot) : 17
3. Données collectées par capteur : 18
4. Implémentation des mesures captées dans le serveur RMI : 19
5. Visualisation sur phpMyAdmin : 19
6. Visualisation sur l'interface avec les mesures et les graphiques : 20
7. Exportation des données au format Excel : 21
8. Configuration des seuils des alertes et du courriel de destination : 22
9. Simulation : 23
IV. PROBLEMES RENCONTRES : 26
V. POINTS D'AMELIORATION : 26
VI. CONCLUSION : 26
CONCLUSION GENERALE 27
REFERENCES 28

TEBLE DES FIGURES

Figure 1 : Diagramme de classe 10
Figure 2 : Diagramme de déploiement 11
Figure 3 : Diagramme de séquence 12
Figure 4 : Diagramme des cas d'utilisation 13
Figure 5 : VS Code 14
Figure 6 : Spring Boot 14
Figure 7 : JAVA RMI 15
Figure 8 : MQTT 15
Figure 9 : WOKWI 15
Figure 10 : XAMPP 16
Figure 11 : MySQL 16
Figure 12 : GitHub 16
Figure 13 : Démarrage de XAMPP 17
Figure 14 : Démarrage de Spring Boot 17
Figure 15 : Capteur DHT22 (WOKWI) et MQTT Broker 18
Figure 16 : Implémentation des mesures dans le serveur RMI 19
Figure 17 : PhpMyAdmin 20
Figure 18 : L'interface WEB 21
Figure 19 : Option d'exportation des données 21
Figure 20 : Fichier Excel exporté 22
Figure 21 : Configuration du système 22
Figure 22 : Nouveaux mesures 23
Figure 23 : PhpMyAdmin 23
Figure 24 : L'affichage sur l'interface 24
Figure 25 : Fichier Excel 25
Figure 26 : Les courriels d'alerte 25

RESUME

Dans ce projet, nous avons développé un système pour surveiller la température et l'humidité dans un entrepôt, destinée aux entreprises de stockage et de logistique. Cette solution permet de suivre en temps réel les conditions environnementales, de contrôler les capteurs intégrés et d'envoyer des notifications si les valeurs dépassent les limites établies. Nous avons utilisé une méthodologie agile pour organiser les phases de conception et de développement. L'interface utilisateur a été conçue avec HTML, CSS et JavaScript, tandis que le Framework Java Spring Boot a été employé pour développer la logique métier côté serveur et faciliter la gestion des requêtes. La communication côté serveur repose sur Java RMI. Les données des capteurs sont transmises via le protocole MQTT et enregistrées dans une base de données SQL.

Le suivi des versions a été assuré par Git. Ce projet a renforcé nos compétences en développement full-stack, en communication client-serveur, en gestion de bases de données et en travail collaboratif.

Mots-clés : Java RMI, Spring Boot, SQL, MQTT, HTML, CSS, JavaScript, Git, surveillance, température, humidité, entrepôt.

ABSTRACT

In this project, we developed a system for monitoring temperature and humidity in a warehouse, designed for storage and logistics companies. This solution allows for real-time monitoring of environmental conditions, control of integrated sensors, and sending notifications if values exceed established limits. We used an agile methodology to organize the design and development phases. The user interface was designed with HTML, CSS, and JavaScript, while the Java Spring Boot framework was used to develop server-side business logic and facilitate query management. Server-side communication is based on Java RMI. Sensor data is transmitted via the MQTT protocol and stored in an SQL database.

Version tracking was handled by Git. This project strengthened our skills in full-stack development, client-server communication, database management, and collaborative working.

Keywords: Java RMI, Spring Boot, SQL, MQTT, HTML, CSS, JavaScript, Git, monitoring, temperature, humidity, warehouse.

INTRODUCTION GENERALE

La réalisation de ce projet représente une étape clé dans notre formation, nous permettant d'appliquer nos connaissances théoriques à un problème pratique.

La gestion des conditions environnementales dans les entrepôts logistiques est essentielle pour garantir la qualité des produits stockés et optimiser les opérations. Ce projet s'inscrit dans ce contexte en proposant une application dédiée à la surveillance en temps réel de la température et de l'humidité, conçue pour répondre aux besoins des gestionnaires d'entrepôts.

L'objectif principal est de développer un système fiable, capable de collecter, traiter et afficher des données issues de capteurs, tout en alertant les utilisateurs en cas de dépassement des seuils prédéfinis. En adoptant une méthodologie agile, notre équipe a structuré le développement autour d'une interface utilisateur intuitive réalisée avec HTML, CSS et JavaScript, et d'une logique serveur robuste basée sur le Framework Java Spring Boot, avec une communication assurée par Java RMI.

Les données sont transmises via le protocole MQTT et stockées dans une base de données SQL, tandis que le contrôle des versions a été géré avec Git. Ce rapport présente les étapes de conception, les choix technologiques, les résultats obtenus, ainsi que les compétences acquises au cours de ce projet.

Ce rapport se divise en trois chapitres. Le premier chapitre expose la présentation de l'application ainsi que le cahier des charges. Le deuxième chapitre détaille l'analyse et la conception de l'application développée. Enfin, le troisième chapitre met en avant les outils employés pour sa réalisation, accompagnés de captures d'écran de l'application avec leurs descriptions.

CHAPITRE 1 : Contexte Général du Projet

I. Introduction :

Ce chapitre vise à présenter le cadre général du projet de développement d'une application de surveillance de la température et de l'humidité pour les entrepôts logistiques. Il expose les enjeux liés à la gestion des conditions environnementales dans ce secteur, les objectifs principaux du projet, ainsi que le contexte organisationnel et technique dans lequel il s'inscrit. En détaillant les besoins des gestionnaires d'entrepôts et les contraintes techniques, ce chapitre pose les bases pour comprendre les choix réalisés tout au long du développement.

II. Généralités sur le projet :

1. Contexte :

La gestion des entrepôts logistiques nécessite un contrôle rigoureux des conditions environnementales, telles que la température et l'humidité, pour préserver la qualité des produits stockés, qu'il s'agisse de denrées périssables, de produits pharmaceutiques ou de matériaux sensibles. Une mauvaise régulation peut entraîner des pertes financières et des risques pour la sécurité. Ce projet s'inscrit dans un contexte où les solutions numériques permettent d'automatiser et d'optimiser la surveillance en temps réel.

2. Objectif :

L'objectif principal est de concevoir une application fiable et intuitive permettant de surveiller en temps réel les conditions environnementales dans un entrepôt, de gérer les données des capteurs, d'envoyer des alertes en cas de dépassement des seuils prédéfinis, et de fournir une interface utilisateur accessible pour faciliter la prise de décision.

III. Les fonctionnalités de système :

- Surveillance en temps réel de la température et de l'humidité via des capteurs.
- Affichage des données sous forme de graphiques et tableaux via une interface utilisateur développée en HTML, CSS et JavaScript.
- Envoi d'alertes automatiques (par notifications) en cas de dépassement des seuils prédéfinis.
- Gestion des capteurs (configuration, activation/désactivation).
- Stockage et consultation des données historiques dans une base de données SQL.
- Communication client-serveur via Java RMI, avec une logique métier gérée par le Framework Spring Boot.
- Transmission des données des capteurs via le protocole MQTT.

IV. Les différentes classes d'usagers du système :

1. Utilisateur :

- Configure et gère les capteurs (ajout, suppression, calibration).
- Définit les seuils d'alerte pour la température et l'humidité.
- Accède aux rapports détaillés et aux données historiques.
- Gère les droits d'accès des autres utilisateurs.
- Consulte les données en temps réel via l'interface utilisateur.
- Reçoit et visualise les alertes en cas de dépassement des seuils.
- Accède aux données historiques pour analyse, selon les autorisations définies.

V. L'étude de l'existant :

L'analyse des solutions existantes montre que plusieurs systèmes de surveillance environnementale sont disponibles, mais ils présentent des limites : coûts élevés, interfaces peu intuitives, ou dépendance à des infrastructures complexes. Certains utilisent des protocoles comme Zigbee ou LoRa, mais le protocole MQTT a été privilégié dans ce projet pour sa légèreté et sa compatibilité avec les systèmes IoT. Les solutions basées sur des Framework comme Spring Boot sont courantes, mais peu intègrent une interface utilisateur légère en HTML, CSS et JavaScript avec une communication via Java RMI.

VI. Solution :

Le système de surveillance en temps réel de la température et de l'humidité représente une avancée notable pour la gestion des entrepôts. Grâce à l'utilisation du protocole MQTT pour une collecte efficace des données, de Java RMI pour une communication client-serveur robuste et d'une interface web, cette solution offre aux gestionnaires une plateforme complète pour optimiser le contrôle des conditions environnementales.

VII. Conclusion :

Ce chapitre a posé les bases du projet en détaillant son contexte, ses objectifs, ses utilisateurs cibles et ses fonctionnalités. L'étude de l'existant a permis d'identifier les forces et les faiblesses des solutions actuelles, orientant ainsi le choix d'une architecture adaptée. Les sections suivantes aborderont les aspects techniques, les étapes de développement et les résultats obtenus.

CHAPITRE 2 : Analyse et conception de système

I. Introduction :

Ce chapitre présente l'analyse et la conception de l'application de surveillance de la température et de l'humidité pour les entrepôts logistiques. Il détaille les étapes de modélisation utilisées pour structurer le développement, en s'appuyant sur la méthodologie UML (Unified Modeling Language). L'objectif est de décrire les choix de conception, les interactions entre les composants du système et les besoins fonctionnels, afin d'assurer une implémentation cohérente et efficace.

II. Diagrammes UML :

La modélisation UML est une approche standardisée pour représenter graphiquement les aspects fonctionnels et techniques d'un système. Elle facilite la communication entre les membres de l'équipe, clarifie les besoins des parties prenantes et sert de guide pour le développement. Dans ce projet, UML a été utilisé pour modéliser les structures, les comportements et les interactions du système, garantissant une conception robuste et bien documentée.

1. Diagramme de classe :

Le diagramme de classe décrit la structure statique du système en identifiant les entités principales et leurs relations. Pour ce projet, il inclut des classes telles que (**Mesure**, **RmiServiceClient**, **ConfigurationService**, **EmailService**, et **DonneesController**).

Chaque classe est définie avec ses attributs (par exemple, **temperature**, **humidite**, **capteurName**) et ses méthodes (par exemple, **envoyerMesure**(), **getDernieresMesures**() pour **RmiServiceClient**).

Les relations, comme l'association entre **RmiServiceClient** et **MesureRepository** pour la gestion des données, ou l'implémentation de l'interface **TemperatureService** par **RmiServiceClient**, sont également représentées, illustrant les interactions et dépendances au sein du système.

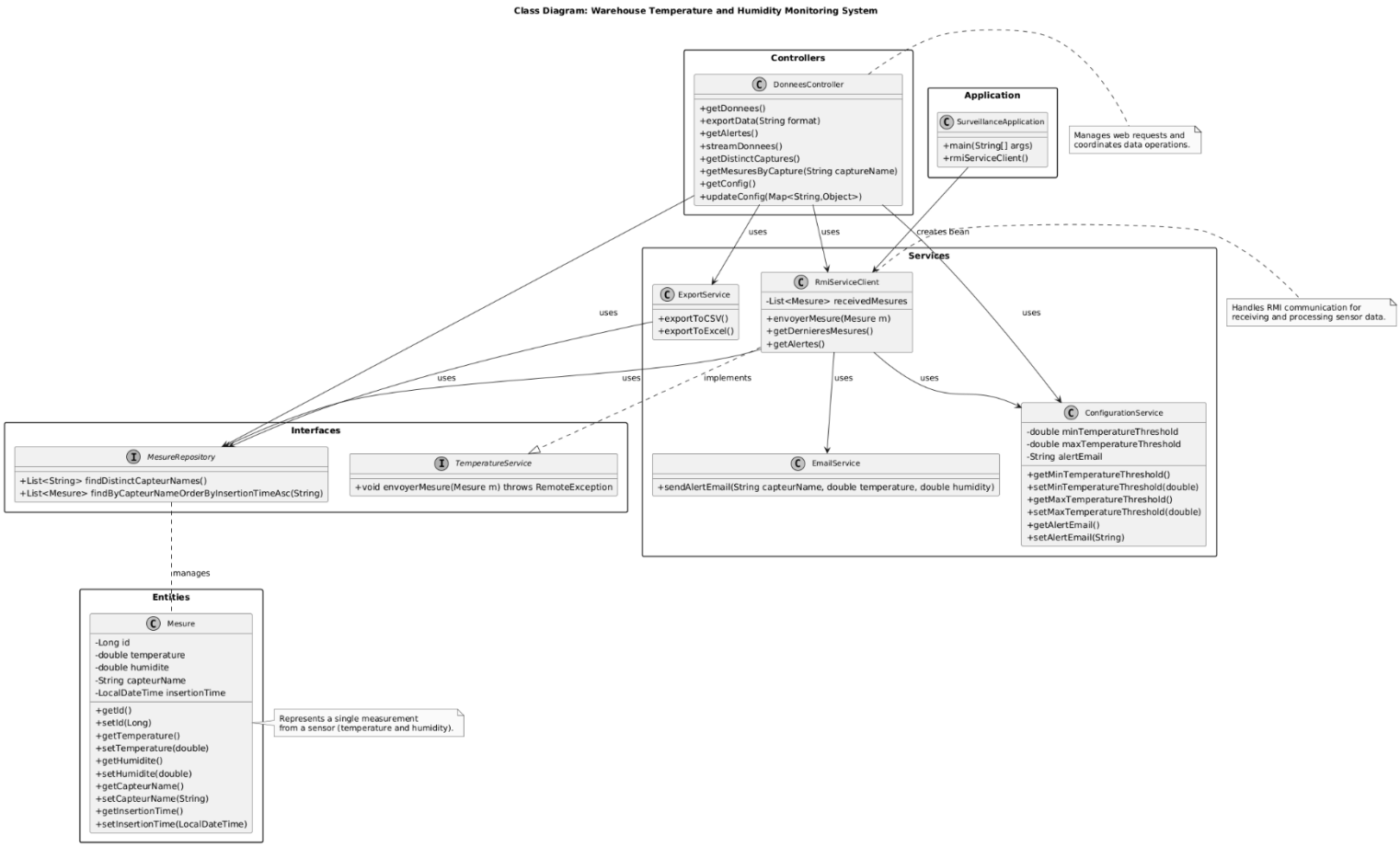


Figure 1 : Diagramme de classe

2. Diagramme de déploiement :

Le diagramme de déploiement illustre l'architecture physique du système, en mettant en évidence les composants matériels et logiciels ainsi que leurs interactions. Il comprend les nœuds suivants :

- Les dispositifs de capteurs embarqués dans l'entrepôt, équipés de firmware intégrant un client MQTT pour transmettre les données de température et d'humidité.
- Une passerelle IoT hébergeant un broker MQTT pour relayer les données des capteurs.
- Un serveur d'application exécutant l'application Spring Boot, qui inclut les composants **DonneesController**, **RmiServiceClient**, **ConfigurationService**, **EmailService** et **ExportService**, ainsi qu'un registre Java RMI pour la communication à distance.
- Un serveur de base de données hébergeant une base SQL, gérée via **MesureRepository**, pour stocker les mesures et configurations.
- Les dispositifs clients (par exemple, PC ou mobiles) exécutant un navigateur web pour accéder à l'interface utilisateur via HTTP/REST.
- Les connexions réseau incluent la communication MQTT entre les capteurs et le broker, les interactions JDBC entre l'application Spring Boot et la base de données, les appels

Java RMI pour les services distants, et les échanges HTTP/REST entre le navigateur et l'application. Ce diagramme clarifie la répartition des composants et leurs interconnexions, offrant une vue d'ensemble de l'architecture déployée.

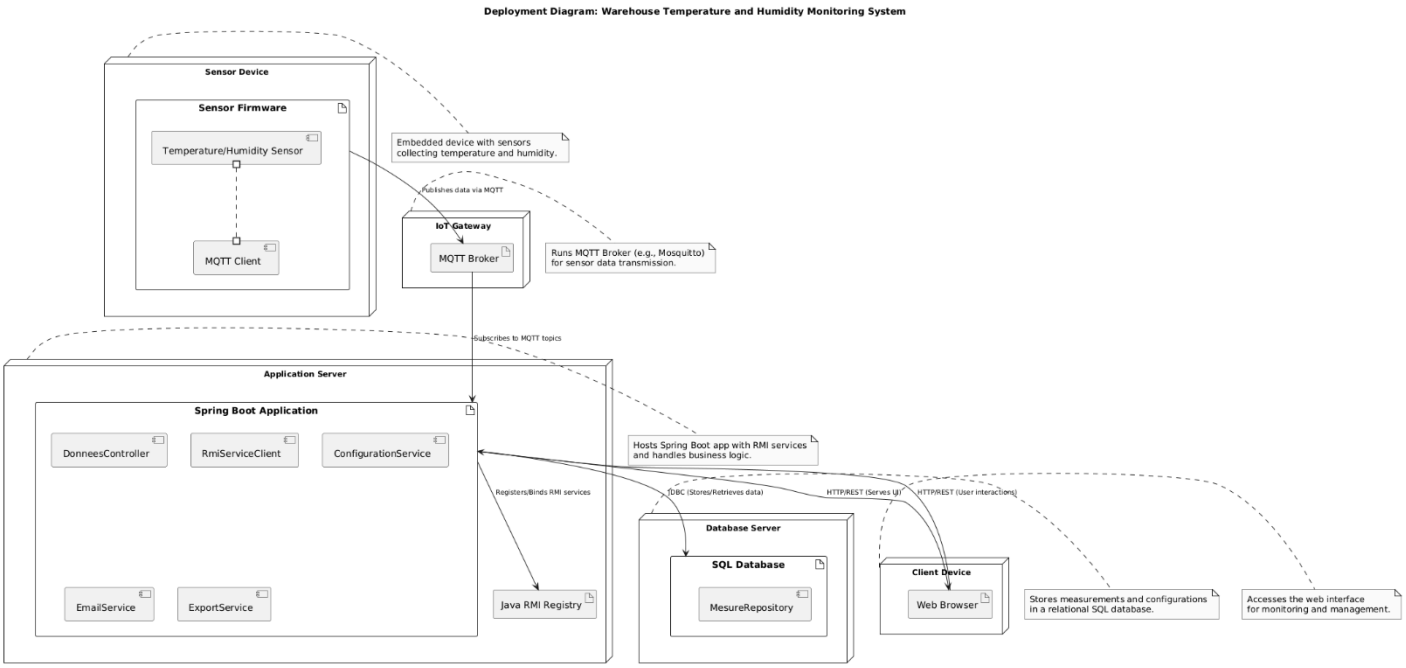


Figure 2 : Diagramme de déploiement

3. Diagramme de séquence :

Le diagramme de séquence illustre les interactions dynamiques entre les composants du système pour des scénarios clés. Pour le scénario « Déclenchement d'une alerte » :

- Le capteur, via son firmware, publie les données de température et d'humidité au broker MQTT.
- Le broker MQTT transmet ces données à **RmiServiceClient** dans l'application Spring Boot, qui est abonnée au topic MQTT.
- **RmiServiceClient** enregistre les données via **MesureRepository** dans la base de données SQL et interroge **ConfigurationService** pour vérifier si les seuils de température ou d'humidité sont dépassés. Si un seuil est dépassé, **RmiServiceClient** déclenche **EmailService** pour envoyer une alerte par email à l'adresse configurée.
- Parallèlement, l'administrateur, via un navigateur web, envoie une requête HTTP à **DonneesController** pour consulter les données, qui récupère les mesures via **RmiServiceClient** et **MesureRepository** pour les afficher.

Ce diagramme met en évidence l'ordre chronologique des opérations et les interactions entre le capteur, le broker MQTT, l'application Spring Boot, la base de données SQL, et l'interface

utilisateur, en intégrant MQTT pour la collecte des données, Java RMI pour la communication serveur, et HTTP/REST pour l'interaction avec l'utilisateur.

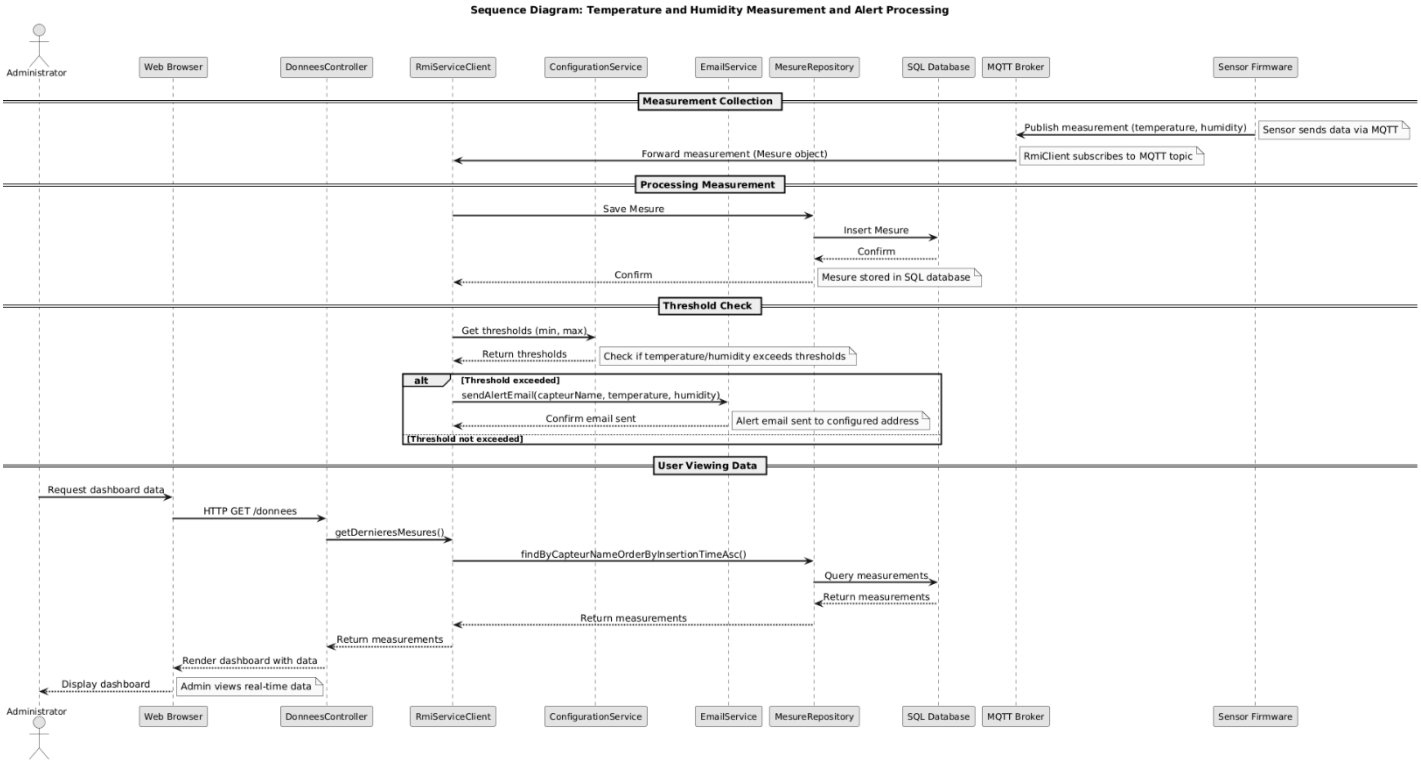


Figure 3 : Diagramme de séquence

4. Diagramme des cas d'utilisation :

Le diagramme de cas d'utilisation illustre les interactions entre les acteurs (Utilisateur, Dispositif Capteur) et le système. Les cas d'utilisation principaux incluent :

- Pour l'Utilisateur : visualiser les données en temps réel, exporter les données, consulter les alertes, configurer les seuils, diffuser les données en continu, gérer les capteurs, gérer les utilisateurs, et envoyer des mesures.
- Pour le Dispositif Capteur : transmettre les mesures de température et d'humidité au système.

Ce diagramme met en évidence les fonctionnalités essentielles du système, en regroupant toutes les responsabilités sous l'acteur Utilisateur, et reflète la dépendance du Dispositif Capteur pour fournir les données initiales.

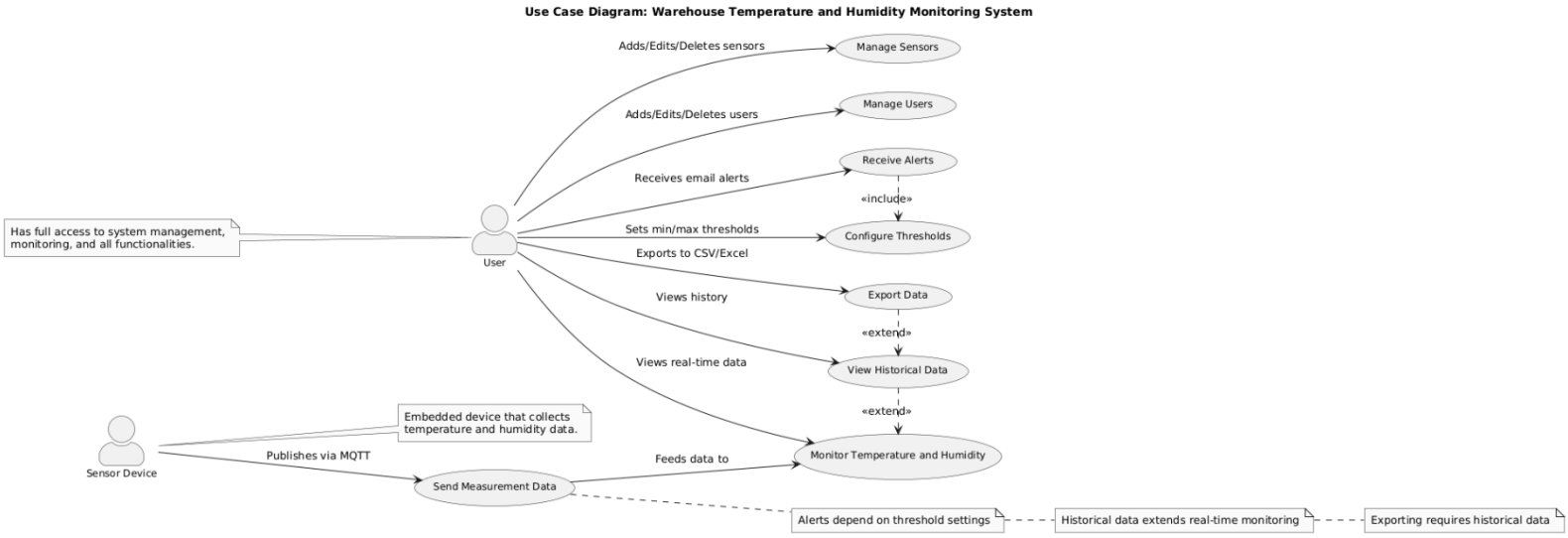


Figure 4 : Diagramme des cas d'utilisation

III. Conclusion :

Ce chapitre a détaillé l'analyse et la conception de l'application à travers la modélisation UML, en couvrant les aspects statiques, fonctionnels, physiques et dynamiques du système. Les diagrammes produits ont permis de structurer le développement, d'assurer une compréhension commune des besoins et de poser les bases pour l'implémentation technique. Le chapitre suivant abordera les étapes de développement et les technologies utilisées.

CHAPITRE 3 : Réalisation de système

I. Introduction :

Ce chapitre décrit les étapes concrètes de développement de l'application de surveillance de la température et de l'humidité pour les entrepôts logistiques. Il présente les outils utilisés, les phases de réalisation, ainsi que les tests effectués pour garantir la fiabilité et les performances du système. L'objectif est de détailler comment les concepts définis dans les chapitres précédents ont été traduits en une solution fonctionnelle.

II. Les outils utilisés :

1. Visual Studio Code :

Visual Studio Code (VS Code) a été utilisé comme environnement de développement principal. Cet éditeur léger et personnalisable a facilité l'écriture, le débogage et la gestion du code pour les différentes technologies employées, notamment HTML, CSS, JavaScript, Java, et SQL, grâce à ses extensions dédiées.

Figure 5 : VS Code

2. Spring Boot :

Spring Boot est un Framework Java open source basé sur Spring et utilisé pour la programmation d'applications autonomes avec un ensemble de bibliothèques qui facilitent le démarrage et la gestion des projets. Spring Boot est une extension du Framework Spring basée sur le principe de convention plutôt que configuration, conçue pour aider à minimiser les préoccupations liées à la configuration lors du développement d'applications Spring.

Le Framework Java Spring Boot a été utilisé pour développer la logique métier côté serveur. Il a permis de structurer l'application, de gérer les requêtes HTTP, et de faciliter l'intégration avec Java RMI pour la communication client-serveur. Spring Boot a également simplifié la configuration de l'application et la connexion à la base de données.

Figure 6 : Spring Boot

3. JAVA RMI :

Java Remote Method Invocation (RMI) a été utilisé pour assurer la communication entre le client (interface web) et le serveur. RMI permet d'appeler des méthodes distantes de manière transparente, facilitant l'échange de données en temps réel, comme les mesures des capteurs et les alertes.

Figure 7 : JAVA RMI

4. MQTT :

MQTT est un protocole de messagerie standard de l'OASIS pour l'Internet des objets (IoT). Il est conçu comme un transport de messagerie publish/subscribe extrêmement léger qui est idéal pour connecter des appareils distants avec une petite empreinte de code et une bande passante minimale de réseau. MQTT est aujourd'hui utilisé dans une grande variété d'industries, comme l'automobile, la fabrication, les télécommunications, le pétrole et le gaz, etc.

Le protocole MQTT (Message Queuing Telemetry Transport) a été employé pour la transmission des données des capteurs embarqués vers le serveur. Ce protocole léger, basé sur un modèle publish/subscribe, est idéal pour les systèmes IoT, garantissant une collecte efficace des données dans l'entrepôt.

Figure 8 : MQTT

5. WOKWI :

Wokwi, une plateforme de simulation pour systèmes embarqués, a été utilisée pour modéliser et tester les capteurs de température et d'humidité. Elle a permis de simuler le comportement des capteurs dans un environnement virtuel, facilitant le développement et la validation du système avant un éventuel déploiement physique.

Figure 9 : WOKWI

6. XAMPP :

XAMPP a été utilisé pour configurer un environnement de développement local intégrant un serveur web Apache et MySQL. Cet outil a facilité la gestion de la base de données MySQL et le test de l'interface web en local, garantissant un déploiement fluide des composants serveur et client avant la mise en production.

Figure 10 : XAMPP

7. MySQL :

MySQL a été choisi comme système de gestion de base de données relationnelle pour stocker les données des capteurs (température, humidité, horodatage) et les configurations des seuils d'alerte. Sa simplicité, sa performance et sa compatibilité avec Spring Boot en font un choix adapté pour ce projet.

Figure 11 : MySQL

8. GitHub :

GitHub a servi de plateforme pour le contrôle de version et la collaboration en équipe. En utilisant Git, les membres ont pu gérer les versions du code, créer des branches pour les fonctionnalités spécifiques, et fusionner les modifications via des pull requests, assurant un développement organisé et traçable.

Figure 12 : GitHub

III. Fonctionnement de projet :

1. Démarrage de XAMPP :

Le processus commence par le lancement de XAMPP pour activer le serveur web Apache et la base de données MySQL. Une fois XAMPP démarré, phpMyAdmin est accessible pour gérer la base de données, créant ainsi un environnement local pour stocker et consulter les données des capteurs.

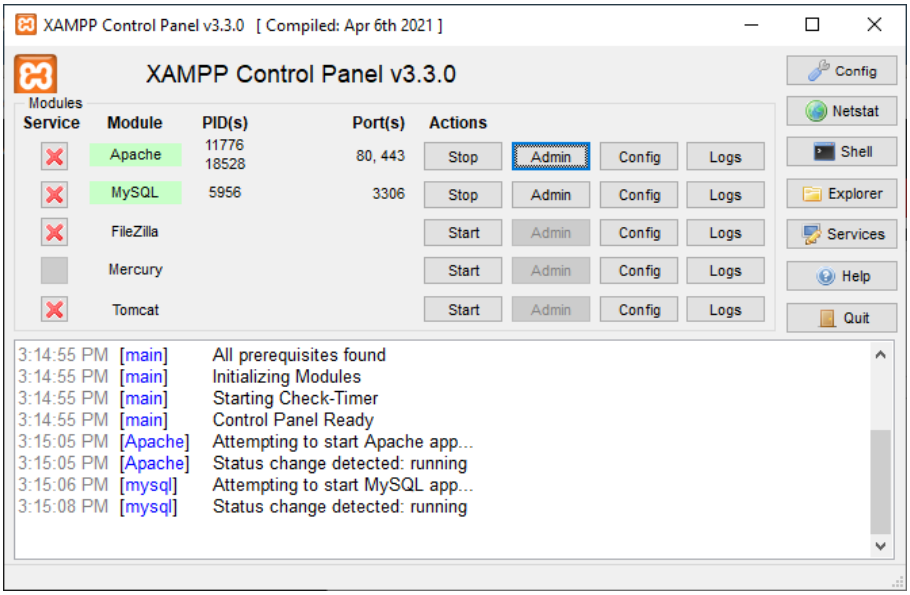


Figure 13 : Démarrage de XAMPP

2. Démarrage de projet (Spring Boot) :

Le serveur Spring Boot est lancé en utilisant la commande `mvn spring-boot:run` dans un environnement configuré avec XAMPP pour MySQL.

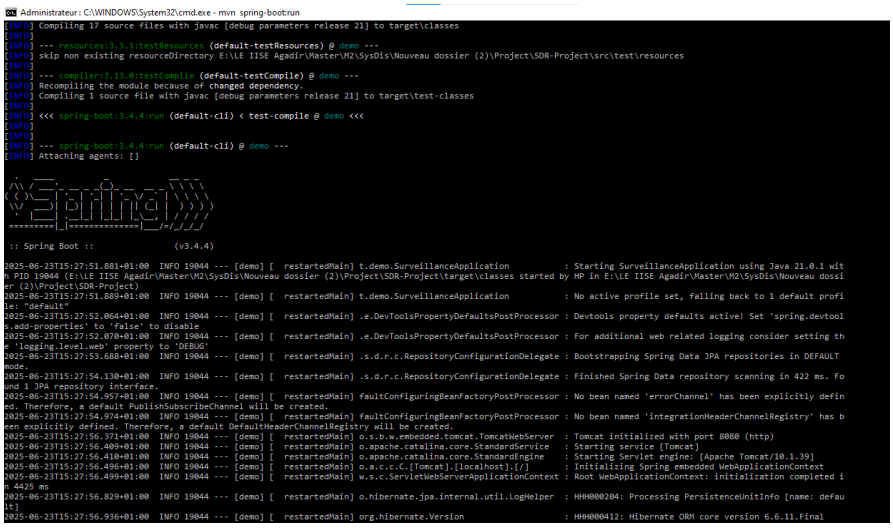


Figure 14 : Démarrage de Spring Boot

3. Données collectées par capteur :

Le fonctionnement commence par la collecte des données de température et d'humidité via un capteur DHT22 qui est contrôlé par une carte programmable ESP32 simulés sur la plateforme Wokwi.

Ces capteurs génèrent des données en temps réel ou simulées, qui sont formatées et publiées vers un broker MQTT (HiveMQ) sous un topic spécifique (esp32/data).

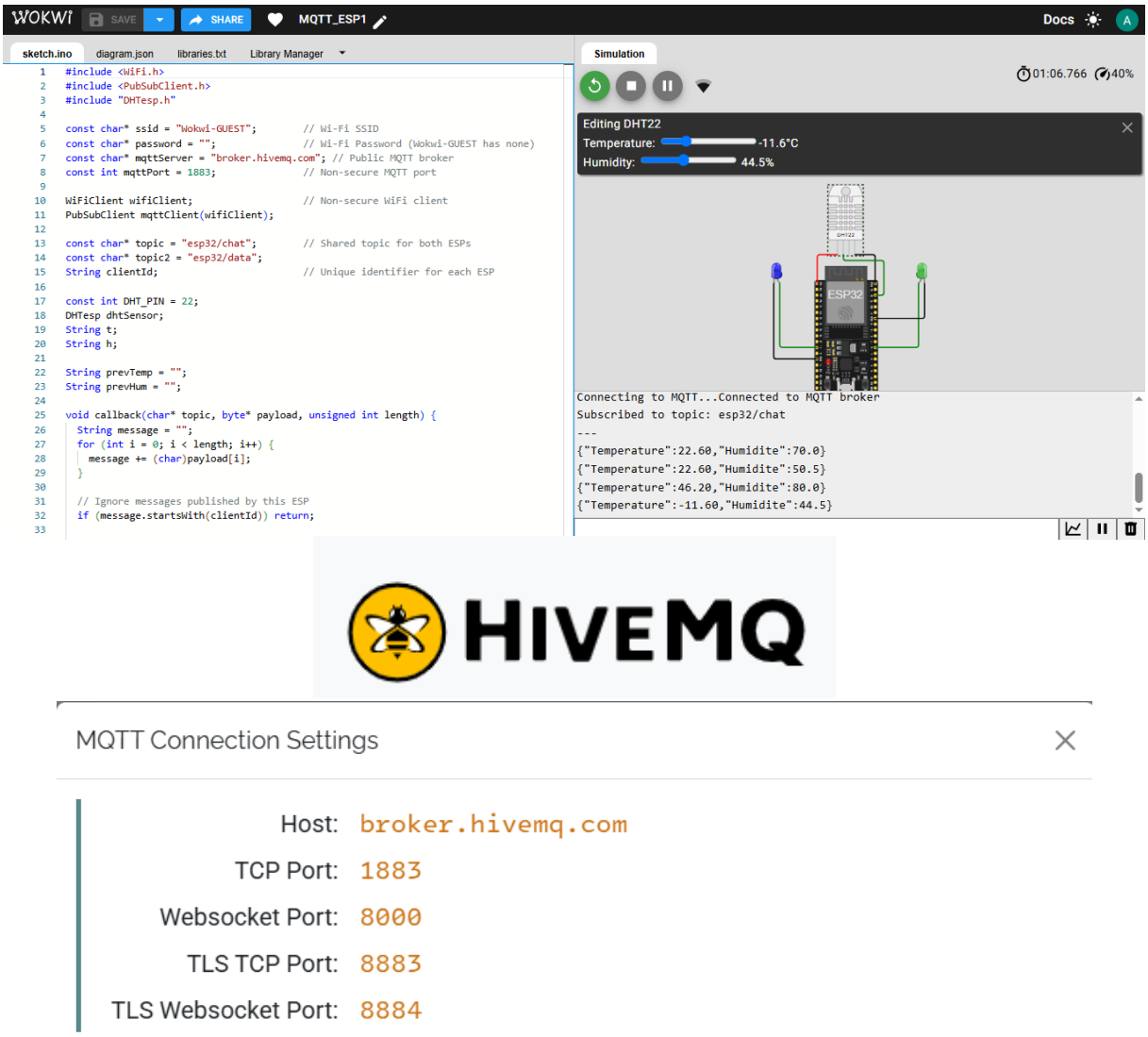


Figure 15 : Capteur DHT22 (WOKWI) et MQTT Broker

4. Implémentation des mesures captées dans le serveur RMI :

Les données issues des capteurs simulés sur Wokwi sont transmises en temps réel via le protocole MQTT au serveur Spring Boot. Ce dernier joue un rôle central dans l'architecture distribuée : il reçoit les messages JSON, les interprète à l'aide d'une classe dédiée qui les convertit en objets métier de type Mesure, puis les valide. Une fois les données traitées, elles sont envoyées au serveur distant à l'aide de Java RMI, qui les expose aux clients via des méthodes accessibles à distance. Ce processus assure une communication fluide et fiable entre les capteurs, le traitement applicatif et l'interface utilisateur.

```
Administrateur : C:\WINDOWS\System32\cmd.exe - mvn spring-boot:run
ntegration.errorLogger} as a subscriber to the 'errorChannel' channel
2025-06-23T15:30:56.777+01:00  INFO 18508 --- [demo] [  restartedMain] o.s.i.channel.PublishSubscribeChannel    : Channel 'demo.errorChannel' has 1 subscriber(s).
2025-06-23T15:30:56.779+01:00  INFO 18508 --- [demo] [  restartedMain] o.s.i.endpoint.EventDrivenConsumer       : started bean '_org.springframework.integration.errorLo
gger'
2025-06-23T15:30:56.779+01:00  INFO 18508 --- [demo] [  restartedMain] o.s.i.endpoint.EventDrivenConsumer       : Adding {message-handler:messageHandler.serviceActivato
r} as a subscriber to the 'mqttInputChannel' channel
2025-06-23T15:30:56.781+01:00  INFO 18508 --- [demo] [  restartedMain] o.s.integration.channel.DirectChannel    : Channel 'demo.mqttInputChannel' has 1 subscriber(s).
2025-06-23T15:30:56.782+01:00  INFO 18508 --- [demo] [  restartedMain] o.s.i.endpoint.EventDrivenConsumer       : started bean 'messageHandler.serviceActivator'
2025-06-23T15:31:08.381+01:00  INFO 18508 --- [demo] [  restartedMain] .m.i.MqttPahoMessageDrivenChannelAdapter : started bean 'inbound'; defined in: 'class path resour
ce [temperature/demo/MQTT/MqttSubscriberConfig.class]'; from source: 'temperature.demo.MQTT.MqttSubscriberConfig.inbound()'
2025-06-23T15:31:08.431+01:00  INFO 18508 --- [demo] [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8080 (http) with context path '
/'
2025-06-23T15:31:08.454+01:00  INFO 18508 --- [demo] [  restartedMain] t.demo.SurveillanceApplication          : Started SurveillanceApplication in 29.244 seconds (pro
cess running for 31.795)
? RECEIVED MQTT MESSAGE!
Topic: esp32/data
Payload: {"Temperature":22.60,"Humidite":50.5}
---
Temperature: 22.6
Humidity: 50.5
Hibernate: insert into mesures (capteur_name,humidite,insertion_time,temperature) values (?,?,?,?)
Nouvelle mesure reçue : 22.6°C / 50.5%
Mesure envoyée !
? RECEIVED MQTT MESSAGE!
Topic: esp32/data
Payload: {"Temperature":46.20,"Humidite":80.0}
---
Temperature: 46.2
Humidity: 80.0
Hibernate: insert into mesures (capteur_name,humidite,insertion_time,temperature) values (?,?,?,?)
Nouvelle mesure reçue : 46.2°C / 80.0%
Alert email sent for sensor: Sensor 1
Mesure envoyée !
? RECEIVED MQTT MESSAGE!
Topic: esp32/data
Payload: {"Temperature":-11.60,"Humidite":44.5}
---
Temperature: -11.6
Humidity: 44.5
Hibernate: insert into mesures (capteur_name,humidite,insertion_time,temperature) values (?,?,?,?)
Nouvelle mesure reçue : -11.6°C / 44.5%
Alert email sent for sensor: Sensor 1
Mesure envoyée !
```

Figure 16 : Implémentation des mesures dans le serveur RMI

5. Visualisation sur phpMyAdmin :

Les données traitées par le serveur sont stockées dans la base de données MySQL, gérée via XAMPP. **PhpMyAdmin** permet de visualiser les enregistrements (température, humidité, horodatage) sous forme de tables, offrant une interface simple pour vérifier l'intégrité et l'organisation des données stockées.

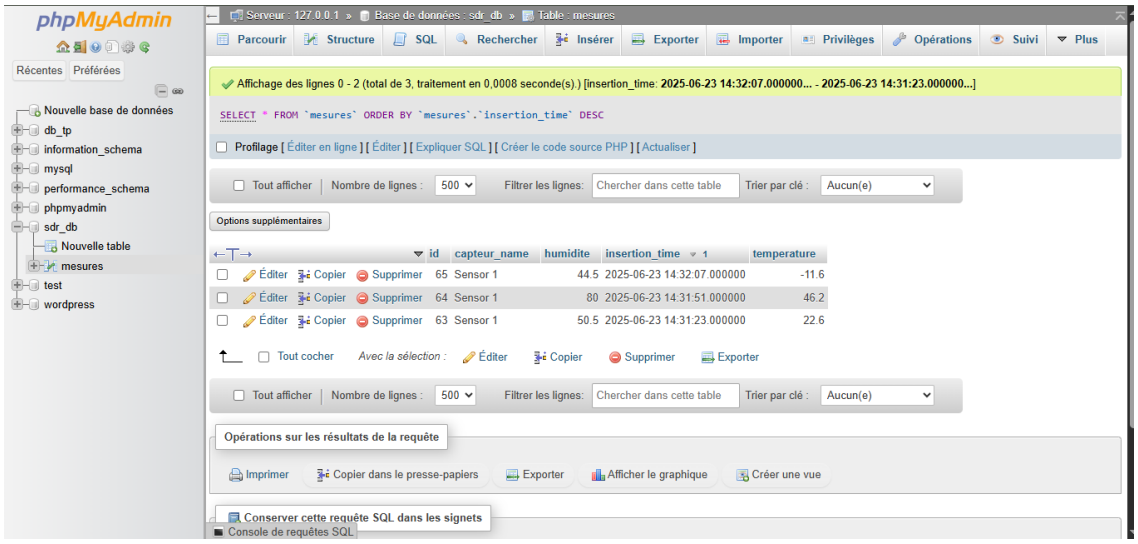
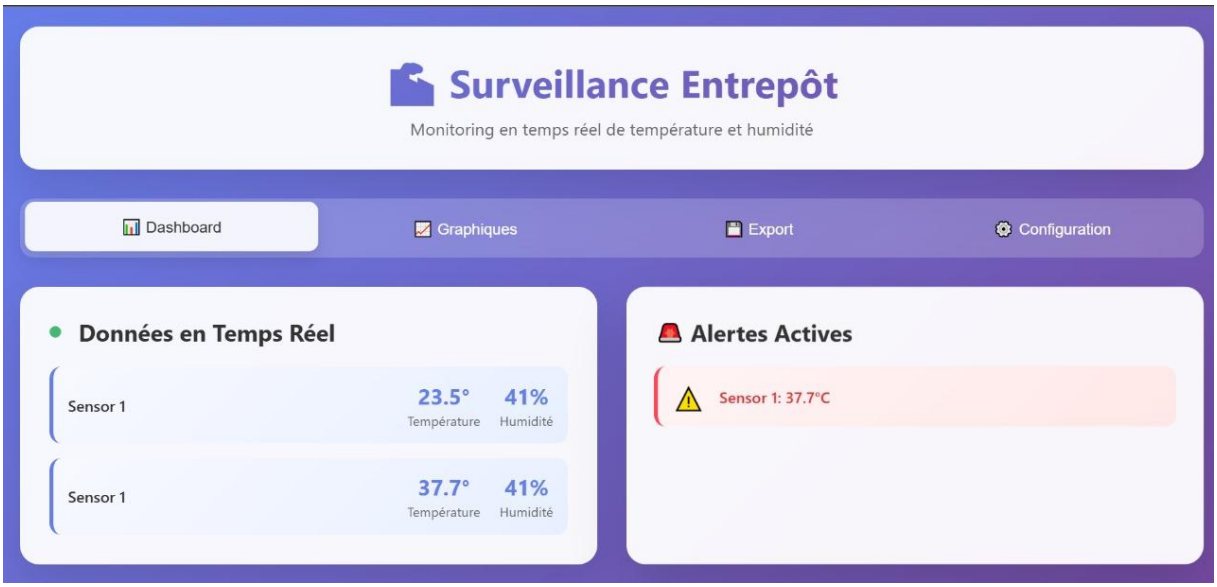


Figure 17 : PhpMyAdmin

6. Visualisation sur l'interface avec les mesures et les graphiques :

L'interface web, développée en HTML, CSS, et JavaScript, récupère les données via des appels RMI au serveur. Elle affiche les mesures en temps réel sous forme des icones ainsi que les alertes en cas de dépassement des seuils prédéfinis, et génère des graphiques dynamiques pour une analyse visuelle des variations de température et d'humidité.



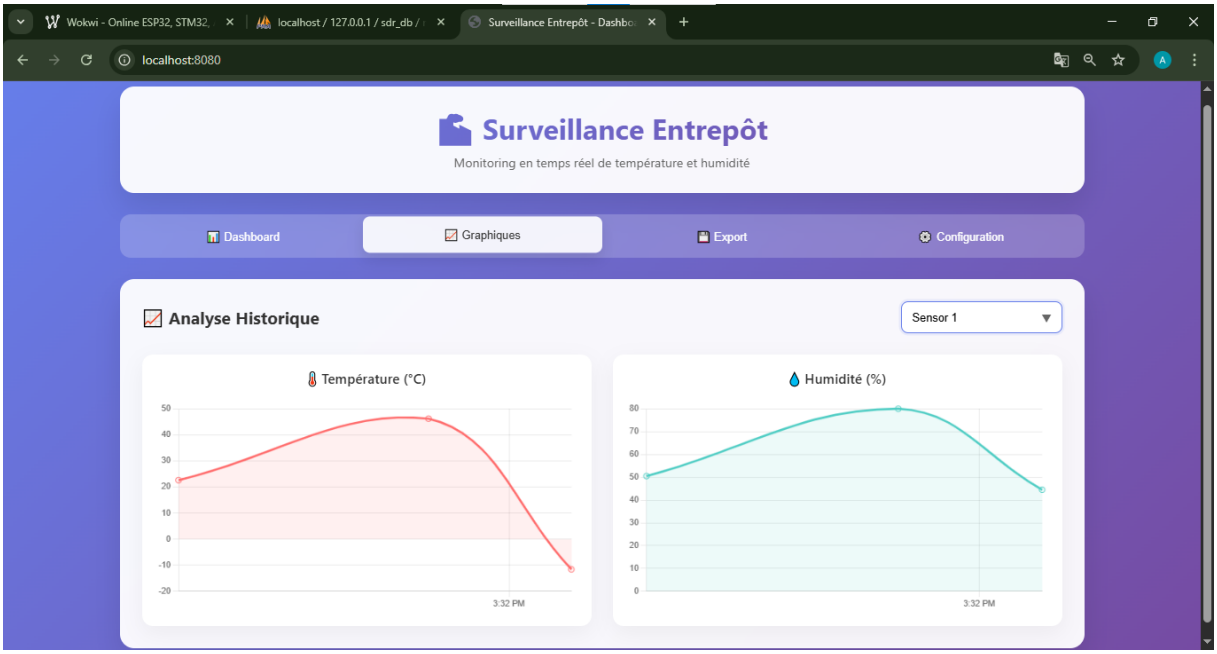


Figure 18 : L'interface WEB

7. Exportation des données au format Excel :

Une fonctionnalité permet d'exporter les données stockées dans MySQL vers un fichier Excel. Cette opération est réalisée via une méthode implémentée dans l'interface JavaScript, qui interroge la base de données via le serveur RMI et génère un fichier téléchargeable contenant les mesures historiques.

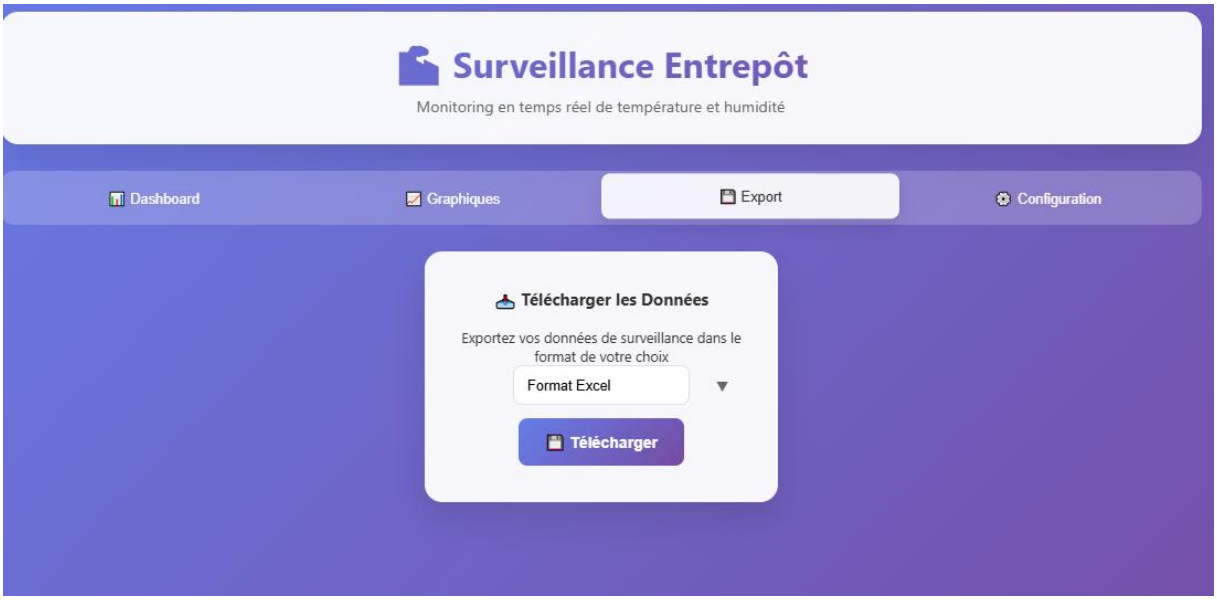


Figure 19 : Option d'exportation des données

	A	B	C	D	E	F
1	ID	Temperature	Humidite	Capteur Name	Insertion Time	
2	63	22.6	50.5	Sensor 1	2025-06-23 15:31:23	
3	64	46.2	80	Sensor 1	2025-06-23 15:31:51	
4	65	-11.6	44.5	Sensor 1	2025-06-23 15:32:07	
5						
6						

Figure 20 : Fichier Excel exporté

8. Configuration des seuils des alertes et du courriel de destination :

L'utilisateur peut configurer les seuils de température et d'humidité via l'interface utilisateur. Ces paramètres sont envoyés au serveur RMI, qui met à jour la base de données MySQL. Une adresse courriel de destination peut également être définie pour recevoir des alertes en cas de dépassement des seuils, implémentées via une bibliothèque Java pour l'envoi d'emails.

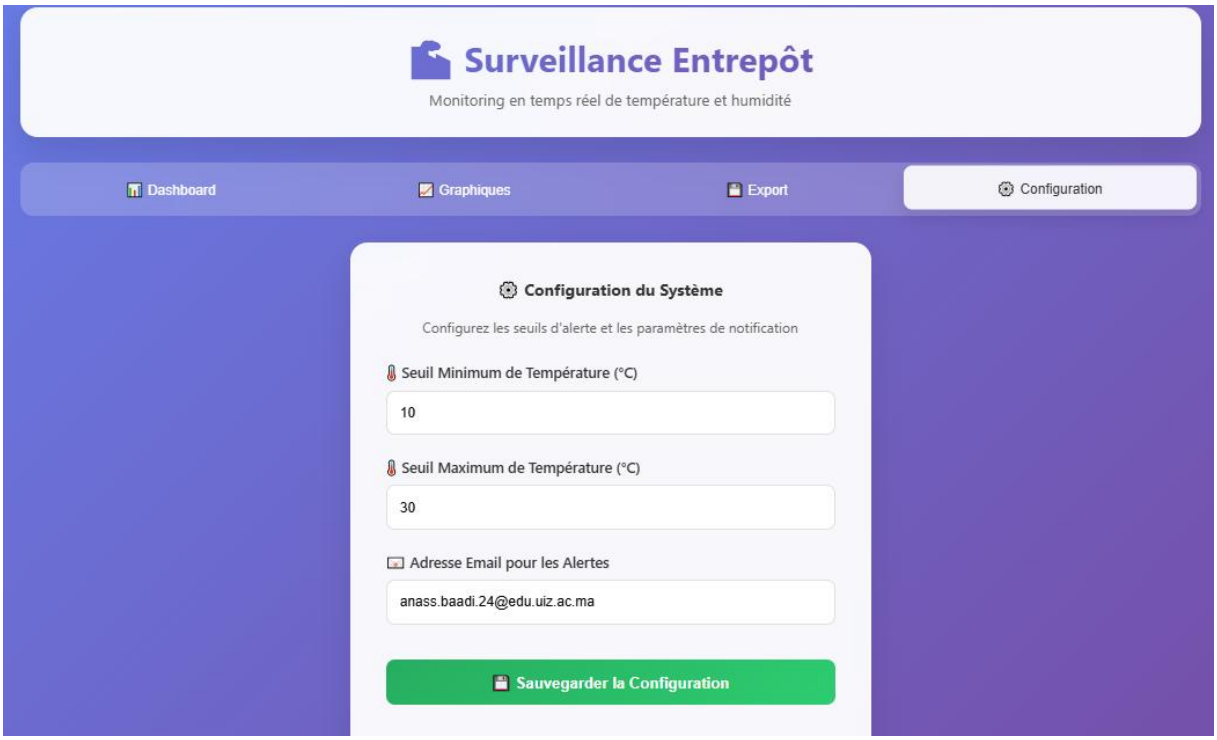


Figure 21 : Configuration du système

9. Simulation :

Nous avons effectué des tests en modifiant les valeurs simulées sur Wokwi (par exemple, en augmentant la température au-delà du seuil). Le système est évalué pour vérifier la détection des anomalies, le déclenchement des alertes, la mise à jour des graphiques, et l'envoi des notifications par courriel, assurant la fiabilité du fonctionnement global.

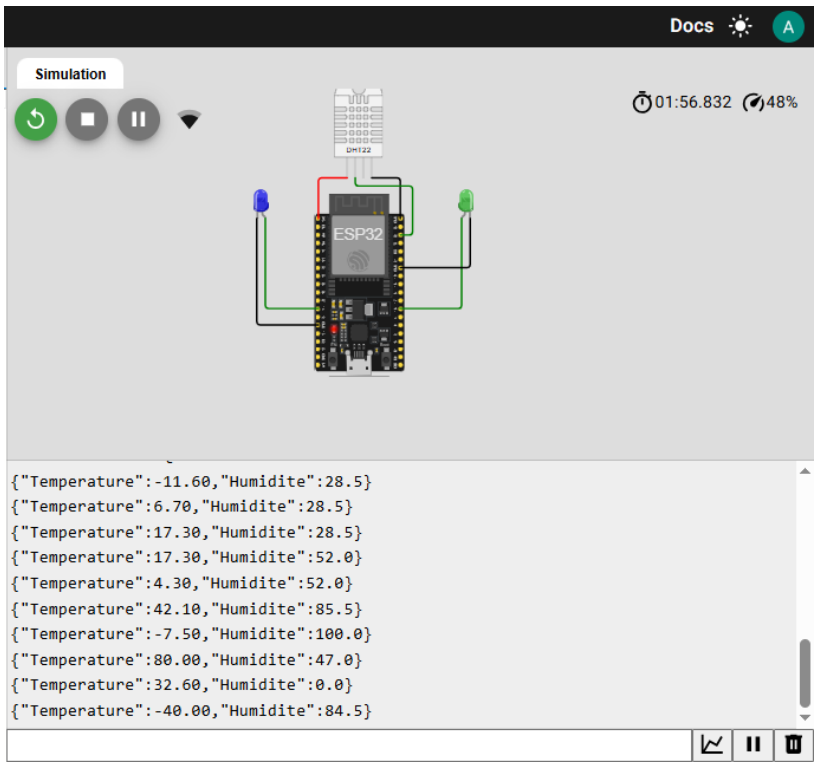


Figure 22 : Nouveaux mesures

– Visualisation des nouveaux données stockées sur phpMyAdmin :

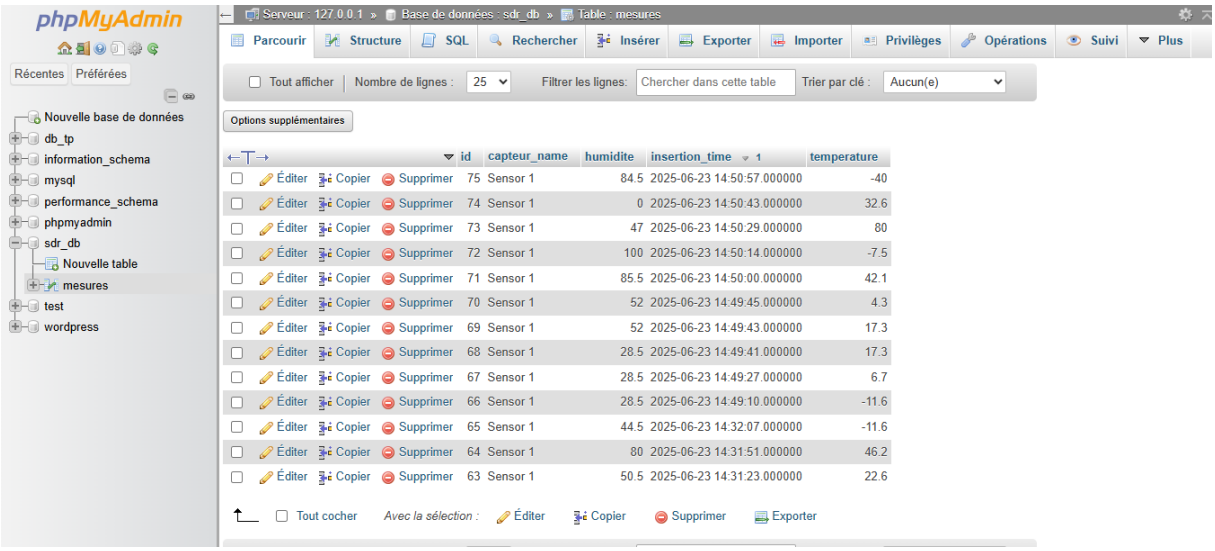


Figure 23 : PhpMyAdmin

– Affichage sur l'interface :

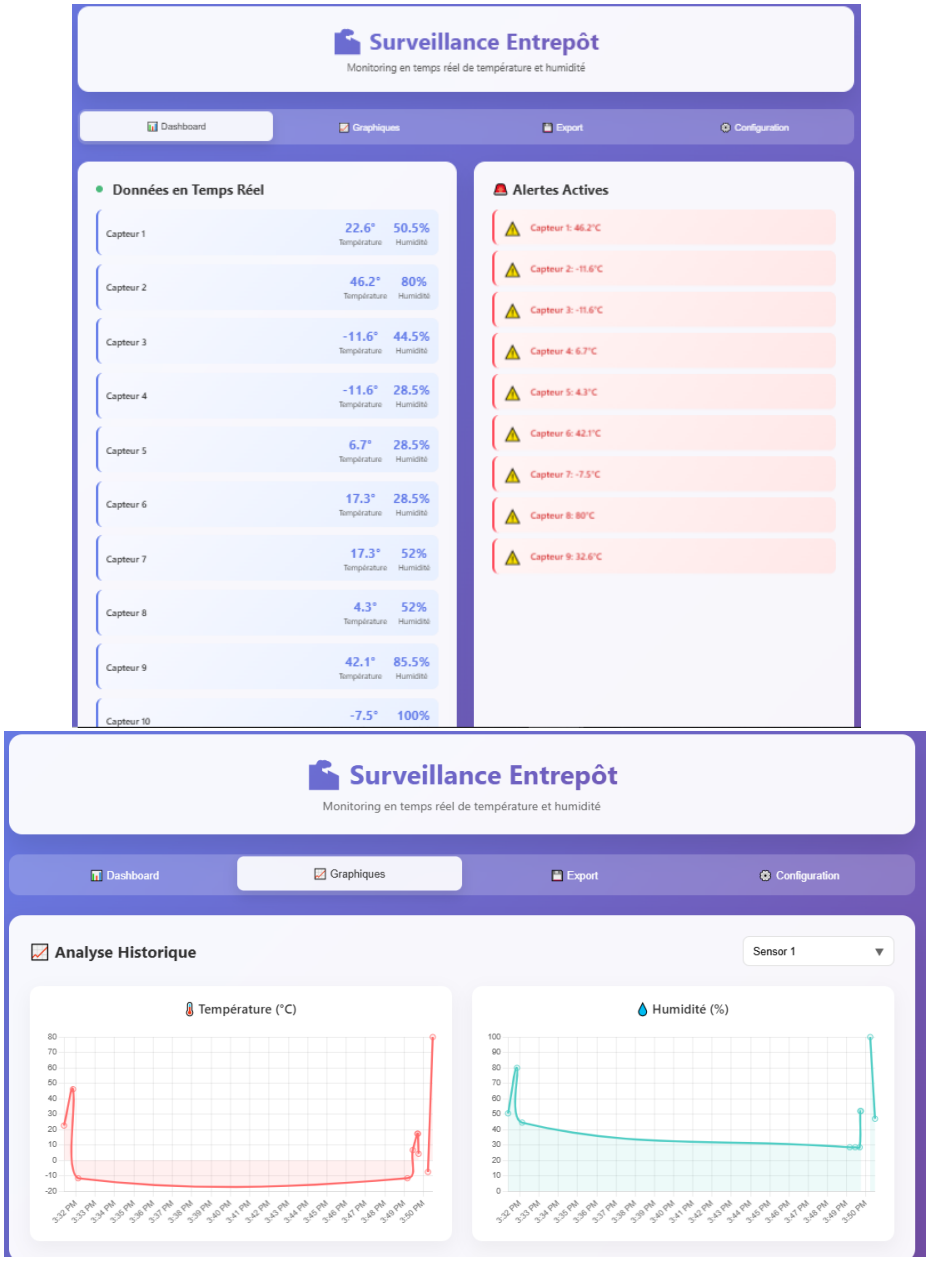


Figure 24 : L'affichage sur l'interface

– Exportation de fichier Excel pour les nouvelles données :

	A	B	C	D	E	F
1	ID	Temperature	Humidite	Capteur Name	Insertion Time	
2	63	22.6	50.5	Sensor 1	2025-06-23 15:31:23	
3	64	46.2	80	Sensor 1	2025-06-23 15:31:51	
4	65	-11.6	44.5	Sensor 1	2025-06-23 15:32:07	
5	66	-11.6	28.5	Sensor 1	2025-06-23 15:49:10	
6	67	6.7	28.5	Sensor 1	2025-06-23 15:49:27	
7	68	17.3	28.5	Sensor 1	2025-06-23 15:49:41	
8	69	17.3	52	Sensor 1	2025-06-23 15:49:43	
9	70	4.3	52	Sensor 1	2025-06-23 15:49:45	
10	71	42.1	85.5	Sensor 1	2025-06-23 15:50:00	
11	72	-7.5	100	Sensor 1	2025-06-23 15:50:14	
12	73	80	47	Sensor 1	2025-06-23 15:50:29	
13	74	32.6	0	Sensor 1	2025-06-23 15:50:43	
14	75	-40	84.5	Sensor 1	2025-06-23 15:50:57	
15						
16						

Figure 25 : Fichier Excel

– Les courriels d'alerte reçois :

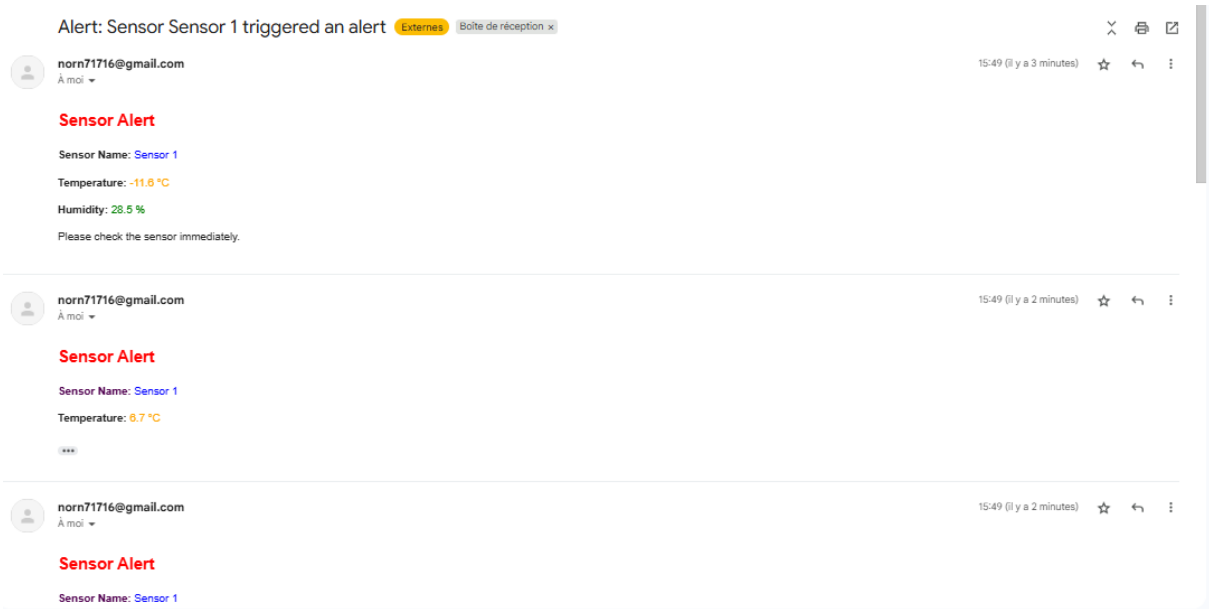


Figure 26 : Les courriels d'alerte

IV. Problèmes rencontrés :

- Latence MQTT : La transmission des données via MQTT a parfois montré des retards, surtout avec un grand volume de données, nécessitant une optimisation du broker.
- Configuration XAMPP : Des conflits de ports ou des erreurs de connexion à MySQL via phpMyAdmin ont été résolus en ajustant les paramètres de XAMPP.
- Incohérences d'environnement entre les capteurs et le serveur (les capteurs qui n'utilise pas JAVA et notre serveur JAVA), nécessitant un outil de communication comme MQTT.

V. Points d'amélioration :

Pour optimiser le système, les axes suivants sont envisagés :

- Amélioration des performances MQTT : Intégrer un broker MQTT plus robuste ou un clustering pour gérer de plus grandes quantités de données.
- Sécurisation des communications : Ajouter une couche de chiffrement (par exemple, TLS) pour les échanges RMI et MQTT.
- Interface utilisateur avancée : Développer une version mobile de l'interface pour une accessibilité accrue.
- Automatisation des tests : Mettre en place des tests unitaires et d'intégration automatisés pour réduire les erreurs humaines.
- Gestion des pannes : Implémenter un mécanisme de redémarrage automatique du serveur en cas de défaillance.

VI. Conclusion :

Ce chapitre a décrit la réalisation de l'application, des outils utilisés au fonctionnement du projet, en incluant les problèmes rencontrés et les points d'amélioration envisagés. L'intégration de technologies comme Spring Boot, Java RMI, MQTT, MySQL, Wokwi, XAMPP, et une interface en HTML, CSS, et JavaScript a permis de répondre aux besoins des gestionnaires d'entrepôts.

CONCLUSION GENERALE

Ce projet, a permis de développer une application de surveillance de la température et de l'humidité pour les entrepôts logistiques, répondant aux besoins des gestionnaires en offrant une solution fiable et intuitive. Grâce à l'intégration de technologies telles que Spring Boot, Java RMI, MQTT, MySQL, Wokwi, XAMPP, et une interface en HTML, CSS et JavaScript, le système permet une collecte en temps réel des données via des capteurs simulés, leur traitement, leur stockage, et leur visualisation sous forme de tableaux, graphiques, et exports Excel.

La configuration des alertes par courriel et les tests fonctionnels ont démontré la robustesse de l'application dans un environnement local. Malgré les défis rencontrés, tels que les latences MQTT, et les problèmes de configuration XAMPP, ces obstacles ont été surmontés, enrichissant les compétences de l'équipe en résolution de problèmes et en gestion de projets.

Les points d'amélioration identifiés, comme l'optimisation des performances MQTT, la sécurisation des communications, et le développement d'une interface mobile, ouvrent des perspectives prometteuses pour une version future plus performante et polyvalente.

En somme, ce projet a non seulement atteint ses objectifs initiaux, mais a également renforcé la collaboration au sein de l'équipe, posant les bases pour une évolution vers une solution industrielle. Il offre ainsi un outil précieux pour la gestion des entrepôts logistiques tout en préparant le terrain pour des innovations futures.

REFERENCES

Anon., s.d. Apache Software Foundation. (2025). Apache Maven Project Documentation..

[Enligne]
Available at: https://maven.apache.org/guides/

Anon., s.d. MQTT.org. (2025). MQTT Version 5.0..

[Enligne]
Available at: http://mqtt.org/documentation

Anon., s.d. Oracle. (2025). Java RMI Documentation..

[Enligne]
Available at: https://docs.oracle.com/javase/8/docs/technotes/guides/rmi/

Anon., s.d. Spring Framework Documentation. (2025). Spring Boot Reference Guide.. [Enligne]
Available at: https://docs.spring.io/spring-boot/docs/current/reference/html/

Anon., s.d. Wokwi. (2025)..

[Enligne]
Available at: https://wokwi.com/docs